Lenguajes Formales y de Programación

Módulo: C++ y Gestión de Memoria

Escuela de Ingeniería de Ciencias y Sistemas - USAC

Adaptación de Contenidos a Lenguajes de Tipado Estático

Agenda del Día

Estructuras

Arreglos Estáticos, Vectores y Matrices en C++

Memoria

Gestión de Structs y Punteros (Stack vs Heap)

Iteración

Bucles clásicos y basados en rangos (Modern C++)

Archivos

Lectura y Escritura con fstream

Competencia

El estudiante comprende el funcionamiento de la gestión de memoria y tipado estático en C++ para la resolución de problemas relacionados con las fases del compilador, contrastando con lenguajes interpretados como JavaScript.

Arreglos: Static vs Dynamic

Tipado Estricto & Memoria

- **Arreglos Estáticos (Raw Arrays):** Tamaño fijo en tiempo de compilación. Se almacenan en el Stack. Muy rápidos, pero inflexibles.
- **std::vector (Dinámicos):** Similar a los Arreglos de JS. Crecen dinámicamente. Almacenan datos en el Heap.
- **Homogeneidad:** A diferencia de JS, en C++ todos los elementos deben ser del *mismo tipo*.

```
// Arreglo Estático (C-Style)
int

// Vector Dinámico (Modern C++)
#include
std::vector int

// Agregar elemento (como push)
      push_back
```

int // Acceso: 2

Arreglos Multidimensionales

Matrices y Tablas

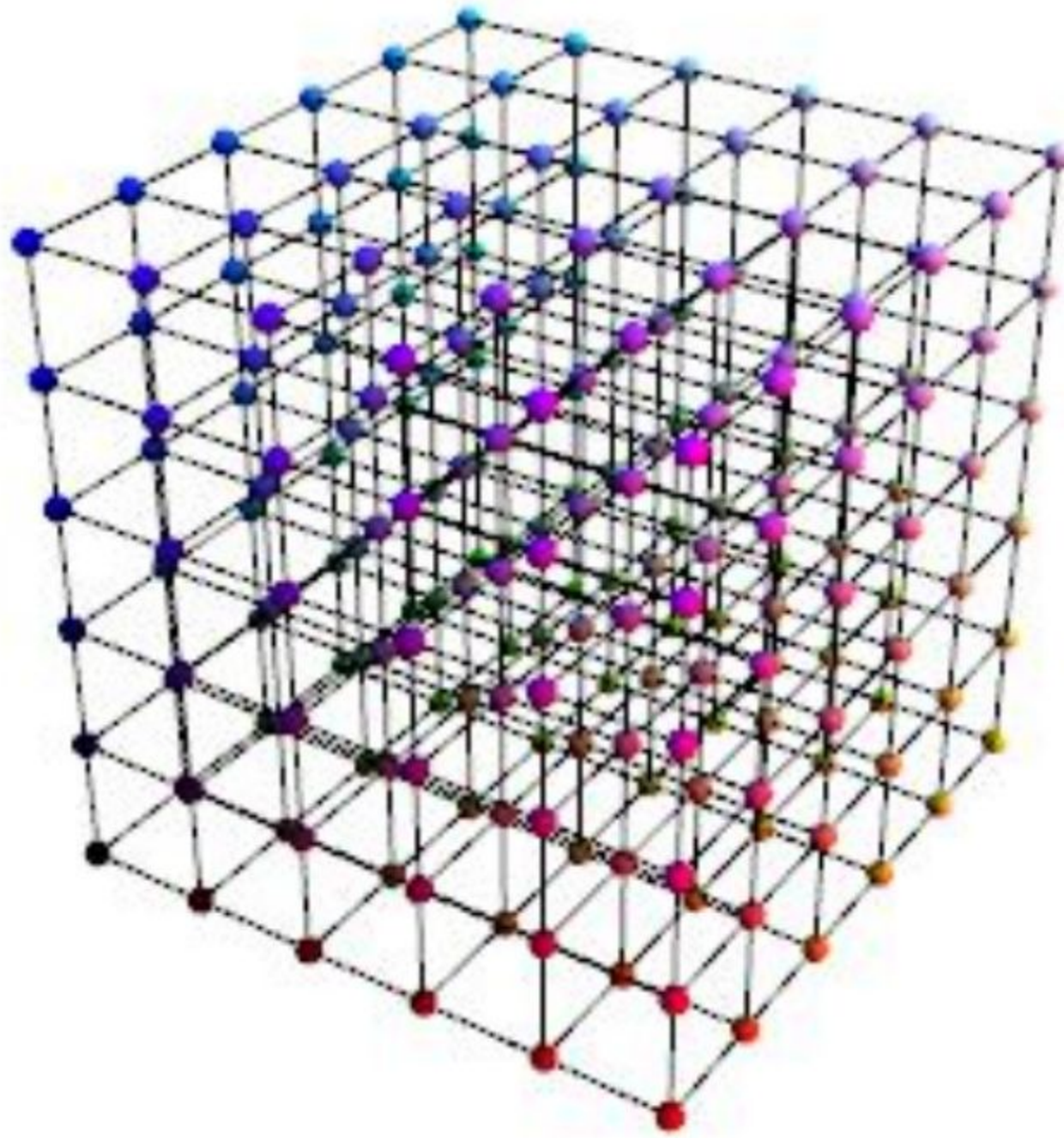
En C++, las matrices pueden representarse como arreglos de arreglos.

Declaración:

```
int matriz[filas][cols];
```

Útil para autómatas, tablas de símbolos o representaciones gráficas en memoria.

int



Acceso y Modificación

El acceso es `O(1)` do índices.

⚠ **Cuidado:** C++ no siempre verifica los límites (bounds checking) en arreglos crudos. Acceder a arr[10] en un arreglo de 5 elementos puede causar un *Segmentation Fault* o leer basura de memoria. Usar .at() en vectores lanza una excepción segura si te pasas del límite.

```
std::vector<int> notas = {85, 90,
78};

// Modificación directa
notas[1] = 95;

// Acceso Seguro vs Inseguro
cout << notas[0];    // Rápido,
sin chequeo
cout << notas.at(0); // Seguro,
con chequeo
```

Strings: std::string

Mutable vs Inmutable

A diferencia de JS, los strings en C++ son **mutables**.

Puedes cambiar un carácter individual directamente.

Para dividir strings (split), no existe un método nativo directo simple, pero usamos stringstream.

```
#include <string>
std::string texto = "Hola Mundo";

// Modificación in-place
texto[0] = 'h';
// texto es ahora "hola Mundo"

// Obtener longitud
int len = texto.length();
```

Estructuras de Datos: Structs

El equivalente a Objetos JS

En JS usamos objetos literales { }. En C++, definimos una plantilla llamada struct o class.

Define la forma de los datos en memoria antes de usarlos. Esto permite que el compilador sepa exactamente cuántos bytes reservar.

```
struct Estudiante {
    std::string nombre;
    int carnet;
    double promedio;
};

Estudiante e1;
e1.nombre = "Juan Perez";
 e1.carnet = 202012345;
```

Diccionarios: std::map

Clave - Valor

Si necesitamos búsquedas rápidas por una clave que no es un índice entero (como un string), usamos std::map.

Similar al Map de JS o un objeto JSON usado como diccionario.

Internamente suele ser un Árbol Binario (Red-Black Tree).

```
#include <map>
std::map<string, int>
inventario;

inventario["Manzanas"] =
50;
inventario["Peras"] = 30;

if
(inventario.count("Manzanas
")) {
    cout << "Hay manzanas";
  }
```

Iteración Clásica

Control Total

Los bucles for y while tienen la misma sintaxis que en JavaScript.

Son fundamentales cuando necesitamos manipular el índice directamente (ej. acceder a i y i+1 simultáneamente).

```
std::vector<int> nums =
{10, 20, 30};

// Bucle For Clásico
for (int i = 0; i <
nums.size(); i++) {
    cout << nums[i] <<
endl;
}

// While
int j = 0;
while (j < nums.size()) {
    // lógica...
    j++;
  }
```

Iteración Moderna (C++11)

Range-based For Loop

Equivalente al for...of de JavaScript.

Es más seguro y limpio. Evita errores de "off-by-one" (salirse del índice por 1).

Usa auto& para referenciar el valor original y evitar copias innecesarias en memoria.

```
std::vector<int> nums =
{10, 20, 30};

// Equivalente a for...of
for (const auto& num :
nums) {
    cout << num << endl;
}

// Para modificar valores
for (auto& num : nums) {
    num *= 2; // Duplica
cada valor
  }
```

Lectura de Archivos

Biblioteca

Usamos la clase ifstream (Input File Stream).

A diferencia de Node.js (asíncrono), la lectura estándar en C++ suele ser síncrona/bloqueante, lo cual es ideal para scripts de procesamiento secuencial.

```
#include <fstream>
std::ifstream
archivo("datos.txt");
std::string linea;

if (archivo.is_open()) {
    while (getline(archivo,
linea)) {
        cout << linea << endl;
    }
    archivo.close();
  }
```

Escritura de Archivos

Output File Stream

Usamos la clase ofstream.

Si el archivo no existe, lo crea. Si existe, por defecto lo sobrescribe a menos que usemos el flag std::ios::app (append) para agregar al final.

```
#include <fstream>

// Modo escritura simple
std::ofstream salida("log.txt");

if (salida.is_open()) {
    salida << "Registro de compilación\n";
    salida << "Error en línea 45\n";

    salida.close();
  }
```

Conceptos Clave

Tipado Estático

Variables y arreglos deben declarar su tipo y respetarlo. int, string, struct.

Memoria

Control manual sobre Stack y Heap. Uso de & (referencias) para eficiencia.

STL

Uso de la Standard Template Library (vector, map, fstream) para no reinventar la rueda.

¿Dudas?

Recuerden consultar el foro semanal.

Próxima clase: Análisis Léxico en C++

Image Sources

https://i.sstatic.net/PEnR1.jpg

Source: stackoverflow.com

https://media.geeksforgeeks.org/wp-content/uploads/20250904151536944343/byte_streams_.webp

Source: www.geeksforgeeks.org

https://png.pngtree.com/png-clipart/20250319/original/pngtree-stylized-3d-question-mark-nobody-abstract-question-mark-vector-png-image_9624876.png

Source: pngtree.com